Unit-2

Stack:

Stack is an ordered collection of homogeneous data elements where the insertion and deletion operations takes place at one end only.

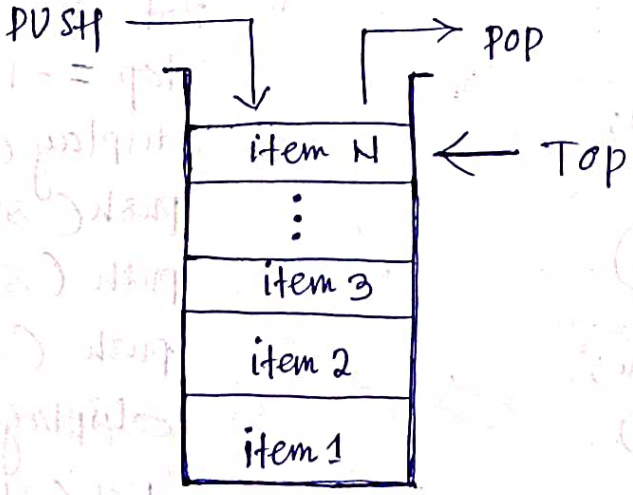
The working principle of stack is Last In First Out (LIFO).

Like array and linked list, stack is also a linear data structure but the only difference is that in case of former two, insertion and deletion operation can take place at any position.

The insertion operation is called PUSH and deletion operation is called POP in stack.

The position where these PUSH and POP operation are performed is called TOP of the stack.

An element in the stack is called ITEM and maximum number of element that can be stored in stack is called SIZE of the stack.

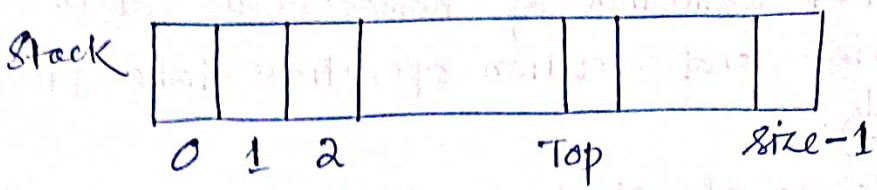


(Illustration of stack)

Generally stack can be represented in memory in two ways:

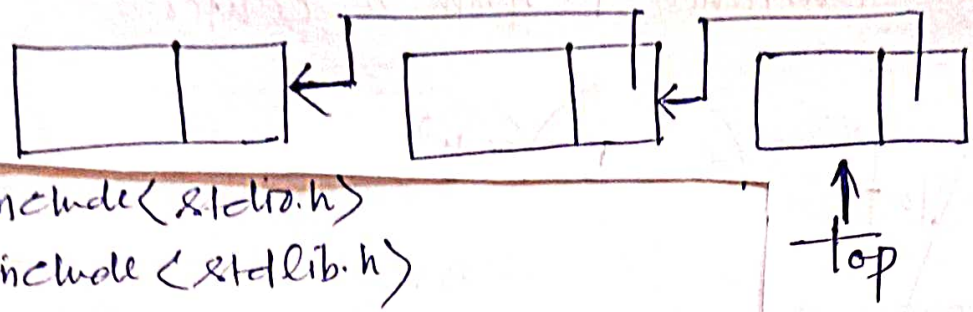
1. Using one dimensional array.
2. Using single linked list.

1. Array representation of stack:




```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int stack[MAX];
int top;
void push(int S[], int item)
{
    if (top == MAX-1)
    {
        printf("\n stack
    is full. Cannot add item\n");
        return;
    }
    top++;
    S[top] = item;
}
void pop(int S[])
{
    int delete_item;
    if (top == -1)
    {
        printf("stack is empty");
        return 0;
    }
    delete_item = S[top];
    top--;
    printf("%d removed from
        stack\n", delete_item);
    return;
}
void display(int S[])
{
    int i=0;
    if (top == -1)
    {
        printf("stack is empty");
        return;
    }
    for(i=top; i>=0; i--)
    {
        printf("\n%d", S[i]);
    }
}
int main()
{
    int item=0;
    top=-1;
    display(stack);
    push(stack, 100);
    push(stack, 200);
    push(stack, 500);
    display(stack);
    pop(stack);
    display(stack);
    push(stack, 700);
    display(stack);
    return 0;
}
```

2. Linked list representation of stack:




```
#include<stdlib.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node * next;
};
void push(int data)
{
    struct Node *temp = (struct Node *)
                malloc(sizeof(struct Node));
    temp->data = data;
    temp->next = top;
    top = temp;
}
void pop()
{
    struct Node *temp;
    if (top == NULL)
    {
        printf("stack is empty");
        return 0;
    }
    else
    {
        temp = top;
        top = top->next;
        temp->next = NULL;
        free(temp);
    }
}
```

```
void display()
{
    struct Node *temp;
    if (top == NULL)
    {  printf ("stack is empty");
       return 0;
    }
    else
    {    temp = top;
         while (temp != NULL)
         {
             printf ("%d\t", temp->data);
             temp = temp-> next;
         }
    }
}
int main()
{   push (100);
    push (200);
    push (300);
    display ();
    pop();
    display();
    push (400);
    display();
    return 0;
}
```

Applications of stack :

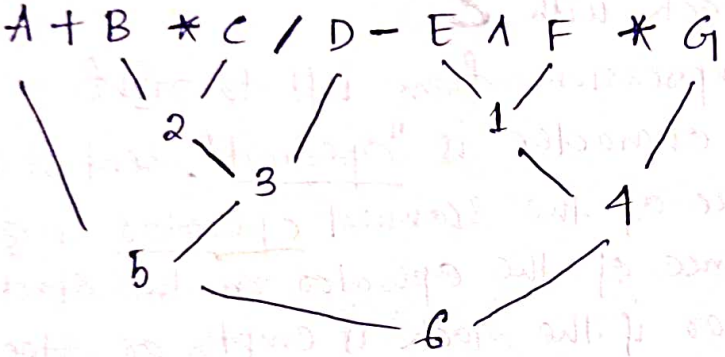
(1) Evaluation of Arithmetic Expressions :

An arithmetic expression consists of operands and operators.

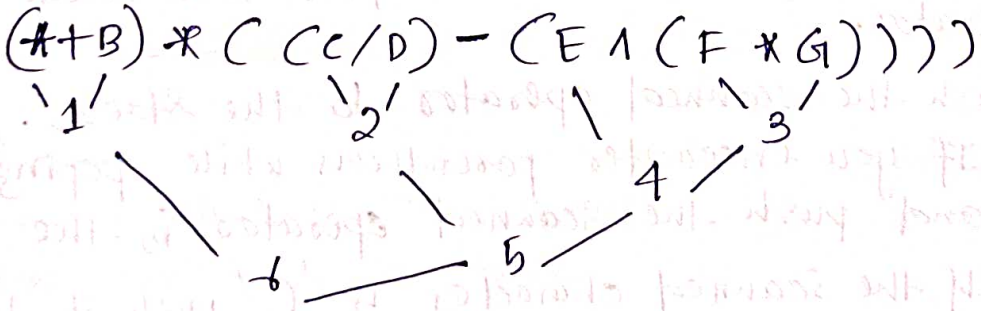
A + B * C / D - E ^ F * G

Expressions are evaluated by precedence and associativity.

Operator	Precedence	Associativity
^	3	Right to left
*, /	2	Left to right
+, -	1	Left to right.



Parenthesize the expression overrides the precedence and associativity.



There are three notations used to represent an arithmetic expression.

1. infix
2. prefix
3. postfix.

The conversional way of writing an expression is called infix.

A+B , C-D, E*F , G/H etc

<operand> <operator> <operand>

Prefix : <operator> <operand> <operand>] Polish Notation.

+AB , -CD, *EF, /GH etc

Postfix : <operand> <operand> <operator>] Reverse Polish Notation

AB+ , CD-, EF*, GH/ etc

Conversion of an infix expression to postfix expression :

Algorithm

Step-① Initialize the stack with (

Step-② Scan the infix expression from left to right.

Step-③ If the scanned character is "operand", sent it to output.

(3.1) If the precedence of the scanned operator is greater than the precedence of the operator in the stack push it into the stack (or if the stack is empty or stack contains a '(' push it into the stack)

(3.2) Else pop all the operators from the stack which are greater than or equal to precedence than the scanned operator.

Push the scanned operator to the stack.

(If you encounter parenthesis while poping stop there and push the scanned operator in the stack).

Step-④ If the scanned character is '(' push it to the stack.

Step-⑤ If the scanned character is ')' pop the stack and output it until a '(' is encountered and discard both '(' & ')'.

Step-⑥ Repeat 2 to 6 until expression is complete.

Step-⑦ Print the output.

Step-⑧ Pop and output from the stack until it is not empty.

Example:

((A + B) ^ C - (D * E) / F)
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

Read Symbol	Stack	Output
Initialize	C	
1. (	C (	
2. A	C (	A
3. +	C (+	A
4. B	C (+	A B
5.)	C	A B +
6. ^	C ^	A B +
7. (	C ^ (	A B + C
8. -	C ^ (-	A B + C
9. (	C ^ (- (	A B + C
10. D	C ^ (- (	A B + C D
11. *	C ^ (- (*	A B + C D
12. E	C ^ (- (*	A B + C D E
13.)	C ^ (-	A B + C D E *
14. /	C ^ (- /	A B + C D E *
15 F	C ^ (- /	A B + C D E * F
16)		A B + C D E * F / -

Evaluation of postfix expression:

Algorithm:

step-① create a stack to store values.

step-② scan the given expression.

(2.1) If the character is operand push into the stack

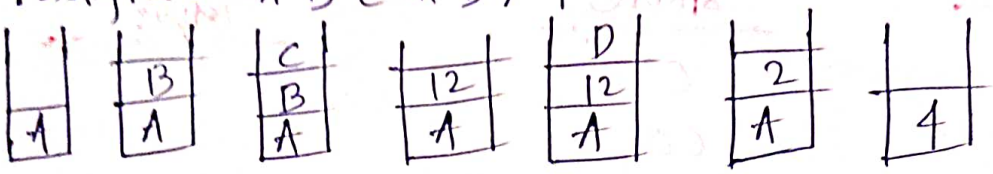
(2.2) If the element is operator
- Pop operands for the operator from the stack
- Evaluate the operator and push result back to the stack

step-③ when the expression is ended, the value in the stack is the final result.

Example : Infix : A + (B * C) / D

A = 2 , B = 3, C = 4 , D = 6

Post-fix : A B C * D / +



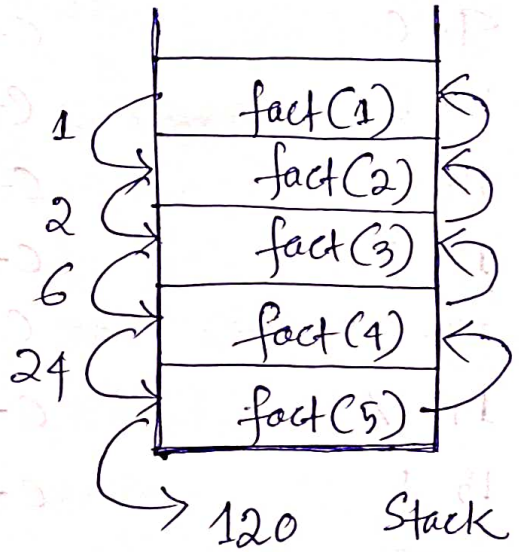
2. Recursions :

The process in which a function calls itself is called recursion, and the corresponding function is called as recursive function.

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 3 \times 2 \times 1$$

$$n! = n \times (n-1)!$$

```
int fact(int n)
{
    if (n<=1)
        return 1;
    else
        return n*fact(n-1);
}
```



Example :

5! = 5 * 4!

4! = 4 * 3!

3! = 3 * 2!

2! = 2 * 1!

1! = 1 * 0!

0! = 1

1! = 1

2! = 2

3! = 6

4! = 24

5! = 120